LAPORAN TUGAS BESAR TEORI BAHASA FORMAL DAN AUTOMATA

Milestone 1 - Lexical Analysis

Oleh

13524014 - Yusuf Faishal Listyardi

13524046 - Farrel Limjaya

13524066 - Nathanael Gunawan

13524070 - A. Fawwaz Azzam Wicaksono

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO & INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

BAB I PENDAHULUAN..... 1
I.1 Pengantar..... 1
I.2 Spesifikasi..... 2
I.2.1 Spesifikasi Program..... 2
I.2.2 Daftar Token Bahasa Pemrograman Arion..... 2
BAB II DASAR TEORI..... 6
II.1 Analisis Leksikal dan Lexer..... 6
II.2 Token dalam Bahasa Pemrograman..... 6
II.3 Deterministic Finite Automata (DFA) pada Lexer..... 7
BAB III PERANCANGAN DAN IMPLEMENTASI..... 9
III.1 Gambaran Umum Program..... 9
III.2 Algoritma Lexer..... 9
III.3 Struktur Direktori Program..... 11
III.4 Implementasi DFA..... 12
III.4.1 State Definition..... 12
BAB IV PENGUJIAN..... 23
IV.1 Uji 1..... 23
IV.2 Uji 2..... 24
IV.3 Uji 3..... 26
IV.4 Uji 4..... 28
IV.5 Uji 5..... 30
IV.6 Uji 6..... 31
IV.7 Uji 7..... 33
IV.8 Uji 8..... 33
BAB V KESIMPULAN DAN SARAN..... 36
V.1 Kesimpulan..... 36
BAB VI LAMPIRAN..... 37
VI.1 Lampiran Link Repository Github..... 37
VI.2 Lampiran Link Workspace Diagram Transisi DFA..... 37
VI.3 Pembagian..... 37

BAB I

PENDAHULUAN

I.1 Pengantar

Apa itu Bahasa Pemrograman? Bahasa pemrograman adalah cara agar kita dapat menuliskan program komputer tanpa perlu menghadapi sulitnya menulis dalam bahasa mesin. Bahasa mesin (biner 1 dan 0) adalah satu-satunya bahasa yang dapat dipahami dan dijalankan oleh CPU, namun sangat sulit dibaca oleh manusia. Oleh karena itu, bahasa pemrograman sering kali dibedakan dengan istilah low-level atau high-level. High-level artinya bahasa lebih mendekati bahasa manusia dan sebaliknya low-level lebih mendekati bahasa mesin. Semakin mendekati bahasa manusia, maka akan semakin banyak proses yang diperlukan untuk mengubahnya menjadi kode mesin.

Terdapat dua macam cara mengubah/memproses bahasa pemrograman menjadi machine code, yaitu compiler dan interpreter. Compiler adalah program yang menerjemahkan seluruh source code menjadi kode mesin (dalam bentuk file e.g. Object File) sebelum program dijalankan. Compiler secara umum terdiri dari 4 tahapan, mulai dari Lexical Analysis, Syntax Analysis, Semantic Analysis hingga Intermediate Code Generation.

Di sisi lain adalah Interpreter yang memproses source code baris per baris secara langsung. Interpreter membaca, menerjemahkan, dan langsung mengeksekusi kode mesin baris per baris sehingga jika terdapat error pada baris ke 10, program akan tetap berjalan normal untuk 9 baris awal, dibandingkan dengan compiler yang akan gagal dijalankan jika terdeteksi terdapat error di dalam source code. Namun dibalik itu, interpreter sebenarnya memiliki tahapan yang sama seperti compiler.

Pada Tugas Besar Teori Bahasa Formal dan Automata, Anda diminta untuk membuat Interpreter untuk bahasa pemrograman Arion, suatu bahasa pemrograman baru yang akan Anda kembangkan. Tugas besar ini dibagi menjadi 4 Milestone, yaitu Lexical Analysis, Syntax Analysis, Semantic Analysis, Intermediate Code Generation dan Interpreter. Setiap Milestone akan memiliki pengumpulannya dan penilaiannya tersendiri.

Pada milestone pertama, Anda diminta untuk membuat tahapan Lexical Analysis dari suatu interpreter.

I.2 Spesifikasi

Pada milestone 1, Anda diminta untuk membuat **lexer atau lexical analyzer** dalam **C/C++** yang menggunakan **Deterministic Finite Automata (DFA)** untuk menghasilkan **token** dari source code bahasa pemrograman Arion.

Deliverables:

- Source Code program,
- Diagram Transisi DFA, dan
- Laporan.

I.2.1 Spesifikasi Program

Dalam milestone ini, Anda diminta untuk membuat **program Lexer** dalam bahasa pemrograman **C/C++ GNU**. Anda **DILARANG menggunakan library atau tools** apapun yang dapat membantu menghasilkan lexical analyzer. Anda wajib menggunakan **MAKEFILE** untuk mempermudah pengembangan dalam bahasa C/C++.

Program Lexer harus berfungsi selayaknya suatu **DFA**. Program Lexer **HANYA BOLEH** membaca **satu demi satu karakter** dari input source code dan memprosesnya sesuai dengan aturan pada DFA. Program harus dapat menerima **input source code** dalam bentuk **.txt**. Program kemudian memproses source code dengan **aturan DFA** dan mengeluarkan **output** daftar token dalam format **.txt**.

Masukan	Kode Sumber (.txt)
Keluaran	Daftar Token (.txt)
Otomata	Deterministic finite automata (DFA)

Catatan Penting: Untuk mempermudah pengerjaan pada milestone-milestone selanjutnya, khususnya pada milestone 4 (pembuatan interpreter), SANGAT DISARANKAN agar program dikembangkan secara modular (terpisah antara komponen Lexer, Semantic, Syntax, dan Intermediate Code Generation). Hal ini supaya komponen dapat dimodifikasi sesuai keperluan tanpa mempengaruhi komponen lainnya, serta dapat dengan mudah dijalankan dalam satu pipeline: lexer - syntax - semantic - intermediate code generation.

I.2.2 Daftar Token Bahasa Pemrograman Arion

Lexer bertujuan untuk mengubah source code dari kumpulan karakter mentah menjadi unit-unit bermakna yang disebut **token**. Setiap token memiliki **jenis (type)** dan **nilai (value)** yang akan digunakan untuk tahap selanjutnya dalam proses kompilasi. Lexer diharapkan dapat menerima source code dan mengeluarkan daftar token yang dihasilkan.

Berikut merupakan daftar token yang harus dapat dibaca oleh lexical analyzer:

Lexer bertujuan untuk mengubah source code dari kumpulan karakter mentah menjadi unit-unit bermakna yang disebut **token**. Setiap token memiliki **jenis (type)** dan **nilai (value)** yang akan digunakan untuk tahap selanjutnya dalam proses kompilasi. Lexer diharapkan dapat menerima source code dan mengeluarkan daftar token yang dihasilkan.

Berikut merupakan daftar token yang harus dapat dibaca oleh lexical analyzer:

Daftar Token			
No	Token	Definisi & Keterangan	Contoh Pemakaian
1	**intcon**	Konstanta integer	1, 3, 48
2	**realcon**	Konstanta bilangan riil	3.14, 26.7
3	**charcon**	Konstanta karakter singular, diapit dengan tanda petik tunggal (‘)	‘j’, ‘k’, ‘t’
4	**string**	Sekuens karakter, diapit dengan tanda petik tunggal (‘)	‘IRK’, ‘TBFO’
5	**notsy**	Operator logika NOT (negasi)	NOT
6	**plus**	Operator aritmatika pertambahan	+
7	**minus**	Operator aritmatika pengurangan	-
8	**times**	Operator aritmatika perkalian	*
9	**idiv**	Operator aritmatika pembagian Integer	div
10	**rdiv**	Operator aritmatika pembagian Riil	/
11	**imod**	Operator aritmatika modulo	MOD
12	**andsy**	Operator logika AND	AND
13	**orsy**	Operator logika OR	OR
14	**eql**	equal	==
15	**neq**	Not equal	<>
16	**gtr**	Greater than	>
17	**geq**	Greater than or equal	>=
18	**lss**	Less than	<
19	**leq**	Less than or equal	<=

20	**lparent**	Left parentheses	(
21	**rparent**	Right parantheses	)
22	**lbrack**	Kurung siku kiri	[
23	**rbrack**	Kurung siku kanan	]
24	**comma**	-	,
25	**semicolon**	-	;
26	**period**	-	.
27	**colon**	-	:
28	**becomes**	Assignment operator	:=
29	**constsy**	Deklarasi konstanta	const
30	**typesy**	Deklarasi tipe data	type
31	**varsy**	Deklarasi variabel	var
32	**functionsy**	Deklarasi fungsi	function
33	**proceduresy**	Deklarasi prosedur	procedure
34	**arraysy**	Deklarasi array	array
35	**recordsy**	Deklarasi record	record
36	**programsy**	Deklarasi program	program
37	**ident**	Nama yang menunjukkan konstanta, tipe, variabel, prosedur, dan fungsi. Nama tersebut harus diawali dengan huruf, yang dapat diikuti oleh kombinasi huruf dan angka apa pun, dan berjumlah berapa pun. Bersifat case-insensitive	x, PI, MyInt, Integer, Real, Char, String h4g, identIniSangatPanjang24tetap iTetapValid
38	**beginsy**	-	begin
39	**ifsy**	-	if
40	**casesy**	-	case
41	**repeatsy**	-	repeat
42	**whilesy**	-	while
43	**forsy**	-	for

44	**endsy**	-	end
45	**elsesy**	-	else
46	**untilsy**	-	until
47	**ofsy**	-	of
48	**dosy**	-	do
49	**tosy**	-	to
50	**downtosy**	-	downto
51	**thensy**	-	then
52	**comment**	Dimulai dengan { atau (* (bukan di dalam tanda kutip) dan diakhiri dengan } atau *).	{ Ini komentar }, (* In komentar *)

BAB II

DASAR TEORI

II.1 Analisis Leksikal dan Lexer

Analisis leksikal merupakan tahap awal dalam proses kompilasi maupun interpretasi bahasa pemrograman. Pada tahap ini, kode sumber yang semula hanya berupa rangkaian karakter mentah dibaca dan diubah menjadi unit-unit yang lebih bermakna. Dengan demikian, tahapan berikutnya tidak lagi bekerja langsung terhadap karakter per karakter, melainkan terhadap hasil pengelompokan karakter yang telah memiliki arti tertentu. Dalam konteks tugas besar ini, milestone pertama memang berfokus pada pembangunan komponen lexical analysis untuk bahasa pemrograman Arion.

Komponen yang menjalankan proses analisis leksikal disebut **lexer** atau **lexical analyzer**. Lexer membaca source code dari kiri ke kanan dan mengenali pola-pola tertentu untuk menentukan jenis token yang sesuai. Selain itu, lexer juga berperan mengabaikan bagian yang tidak diperlukan dalam analisis sintaks, seperti spasi, tab, dan komentar. Keberadaan lexer sangat penting karena hasil tokenisasi akan menjadi masukan utama bagi tahap parsing pada milestone berikutnya. Spesifikasi tugas juga menekankan bahwa program yang dibuat harus berupa lexer dalam bahasa C/C++ GNU dan dikembangkan secara modular agar dapat diintegrasikan dengan tahap syntax, semantic, dan intermediate code generation pada milestone selanjutnya.

Secara umum, tujuan analisis leksikal adalah menyederhanakan proses pemahaman struktur program. Daripada parser memproses karakter mentah satu per satu, parser cukup menerima daftar token yang sudah terstruktur. Oleh sebab itu, perancangan lexer yang benar menjadi fondasi utama dalam pembangunan interpreter Arion secara keseluruhan.

II.2 Token dalam Bahasa Pemrograman

Token adalah satuan terkecil dalam program yang memiliki makna. Setiap token umumnya memiliki **jenis** dan, pada token tertentu, juga memiliki **nilai**. Contohnya, bilangan bulat dapat dikenali sebagai token integer constant, operator penjumlahan dikenali sebagai plus, dan nama variabel dikenali sebagai identifier. Dalam spesifikasi milestone ini, lexer yang dibuat harus mampu menerima source code Arion dan menghasilkan daftar token sebagai output.

Keakuratan pengenalan token sangat menentukan keberhasilan program. Apabila suatu rangkaian karakter salah diklasifikasikan, maka tahap berikutnya akan menerima representasi program yang keliru. Karena itu, lexer harus mampu membedakan secara tepat antara keyword dan identifier, membedakan operator satu karakter dan multi-karakter, serta menangani literal dan komentar sesuai aturan bahasa Arion. Hal ini juga sejalan dengan kriteria penilaian yang menuntut kelengkapan token dan kesesuaian klasifikasi token.

II.3 Deterministic Finite Automata (DFA) pada Lexer

Deterministic Finite Automata (DFA) adalah model automata hingga yang memiliki himpunan state, alfabet input, transisi antarkeadaan, satu state awal, dan satu atau lebih state akhir. Disebut deterministik karena untuk setiap pasangan state dan simbol input, hanya ada satu transisi yang mungkin. Sifat deterministik ini membuat DFA sangat sesuai digunakan dalam lexical analysis, karena proses pembacaan karakter dapat dilakukan secara sistematis dan konsisten. Dalam spesifikasi tugas besar, disebutkan secara eksplisit bahwa lexer Arion harus menggunakan DFA untuk menghasilkan token dari source code.

Pada implementasinya, lexer bekerja seolah-olah merupakan sebuah DFA yang membaca input **satu demi satu karakter**. Setiap karakter yang dibaca akan menentukan perpindahan dari state saat ini ke state berikutnya. Ketika lexer mencapai state akhir yang merepresentasikan suatu pola valid, maka lexer dapat menghasilkan token yang sesuai. Sebagai contoh, rangkaian huruf tertentu dapat berakhir sebagai identifier atau keyword, rangkaian digit dapat dikenali sebagai integer constant atau real constant, sedangkan simbol seperti `:=` atau `>=` dikenali melalui transisi yang mempertimbangkan lebih dari satu karakter. Spesifikasi bahkan menegaskan bahwa program lexer **hanya boleh membaca satu demi satu karakter dari input source code dan memprosesnya sesuai aturan DFA**, sehingga perilaku program benar-benar harus mencerminkan prinsip kerja automata deterministik.

Selain implementasi program, spesifikasi juga mewajibkan pembuatan **diagram transisi DFA** yang sesuai dengan DFA yang digunakan dalam program. Diagram tersebut harus dibuat manual, misalnya menggunakan draw.io, dan tidak boleh dihasilkan secara otomatis dari kode. Kewajiban ini menunjukkan bahwa pemahaman terhadap DFA tidak hanya berhenti pada implementasi, tetapi juga pada kemampuan memodelkan proses pengenalan token secara formal. Oleh karena itu, dalam tugas besar ini DFA berperan

sebagai dasar teoritis sekaligus dasar implementatif bagi pembangunan lexical analyzer Arion.

BAB III

PERANCANGAN DAN IMPLEMENTASI

Implementasi *lexical analysis* diawali dengan perancangan DFA terlebih dahulu. Perancangan DFA ini mempermudah dalam implementasi kode karena proses pembacaan dan pengecekan *lexical analysis* memiliki kemiripan dengan cara bekerja dari DFA yaitu selalu menuju suatu STATE ketika mendapat sebuah input dan hasil *lexical analysis* yang berupa token seperti final STATE pada DFA. Selain itu, cara berpikir selalu maju adalah cara berpikir yang sama dengan DFA.

III.1 Gambaran Umum Program

Gambaran umum program ini pada milestone 1 adalah sebuah lexical analyzer untuk bahasa pemrograman bernama Arion yang bekerja dengan prinsip Deterministic FInite Automata(DFA). Program menerima input masukan berupa file .txt yang berisi source code Arion, kemudian membaca file tersebut satu demi satu karakter dan mengenali token token yang sesuai. Setelah proses pembacaan selesai, program membuat file output .txt yang berisi daftar token yang telah teridentifikasi dari file input

III.2 Algoritma Lexer

Algoritma Lexical Analyzer (Lexer) yang diimplementasikan bekerja dengan membaca karakter dari source code secara berurutan dan mengelompokkannya menjadi token yang bermakna. Berikut adalah alur kerja algoritma lexer dalam menghasilkan token:

1. Inisialisasi dan Pemetaan Keyword

   Saat objek Lexer dibuat, program memanggil initKeywords() untuk memetakan semua kata kunci atau keyword (seperti program, begin, if, var, dll) ke dalam struktur data map. Hal ini memudahkan pencocokan token secara case-insensitive sesuai dengan spesifikasi bahasa Arion

2. Pembacaan Karakter

   Program memproses teks masukan karakter demi karakter untuk mematuhi aturan pembacaan DFA. Fungsi peek() digunakan untuk melihat karakter saat ini tanpa memajukan posisi indeks pembacaan (pos), sedangkan advance() digunakan

untuk membaca karakter saat ini sekaligus menggeser indeks ke karakter berikutnya.

3. Pengabaian Spasi dan Pembacaan Komentar

   Sebelum memproses pengecekan token, pemanggilan getNextToken() akan terlebih dahulu menjalankan fungsi pembantu untuk mengabaikan (skip) karakter whitespace seperti spasi, tab, dan newline. Program juga akan melakukan pembacaan komentar. Komentar merupakan token yang diawali oleh kurung kurawal buka '{' dan diakhiri kurung kurawal tutup '}'. Komentar juga dapat dikenali dari tanda '(*' dan diakhiri oleh '*)'. Token komentar nantinya akan ditampilkan ke output sebagai *comment(isi komentar)*. Komentar yang tidak ditutup (tidak diakhiri tanda kurawal tutup '}' atau tanda '*)' akan dianggap sebagai token UNKNOWN di output.

4. Identifikasi Token

   a. Identifier dan Keyword

      Jika karakter awal adalah huruf, lexer akan terus membaca kombinasi huruf dan angka. String hasil pembacaan kemudian diubah ke huruf kecil (lowercase) dan dicocokkan dengan map keyword. Jika string tersebut terdapat di dalam map, maka token keyword terkait akan dikembalikan. Jika tidak ditemukan, token tersebut diklasifikasikan sebagai ident.

   b. Konstanta Angka (Integer dan Real)

      Jika karakter awal adalah angka, lexer akan membaca seluruh digit yang berurutan. Jika setelah sekumpulan digit ditemukan karakter titik . yang langsung diikuti oleh angka lagi, token diklasifikasikan sebagai konstanta bilangan riil (realcon). Namun, jika tidak ditemukan titik desimal yang valid, token diklasifikasikan sebagai konstanta integer (intcon).

   c. String dan Karakter

      Jika karakter awalan adalah tanda petik tunggal ('), lexer akan membaca sekumpulan karakter hingga menemukan tanda petik penutup. Apabila hanya terdapat tepat satu karakter di dalam tanda petik tersebut, lexer mengklasifikasikannya sebagai token charcon. Sebaliknya, jika terdapat lebih dari satu karakter, kumpulan karakter tersebut diklasifikasikan

sebagai token string. Apabila string tidak diakhiri oleh tanda petik (‘) dan program sudah selesai, akan dianggap sebagai token UNKNOWN dan ditampilkan di output beserta keterangan bahwa string tidak valid.

d. Operator dan Simbol

Simbol dan tanda baca menggunakan struktur switch-case. Beberapa operator yang hanya terdiri dari satu karakter (misalnya +, -, *, /, ;, ,, (,)) akan langsung dikembalikan sebagai token ekuivalen. Untuk operator relasional dan assignment yang berpotensi terdiri dari dua karakter (seperti ==, >=, <=, <>, :=), algoritma akan menggunakan peek() untuk melihat satu karakter ke depan guna menentukan apakah karakter-karakter tersebut harus digabung menjadi satu token ganda atau dipisah.

III.3 Struktur Direktori Program

Struktur directory program kami adalah seperti berikut

```
SMG-Tubes-IF2224-2026/
├── Makefile              # Skrip untuk kompilasi
├── README.md             # File ini
├── bin/                  # Executable File
├── doc/                  # Dokumentasi
├── src/                  # Folder Source Code
│   ├── main.cpp         # Program utama
│   ├── lexer.cpp        # Implementasi lexer
│   └── lexer.hpp        # Header file lexer
└── test/                # Folder Test
    ├── input/           # File input test
    └── output/          # File output hasil run
```

Struktur directory program disusun secara sederhana dan modular agar proses pengembangan, pengujian, dan dokumentasi dapat dilakukan dengan lebih rapi. Pada root

project terdapat file Makefile yang digunakan untuk proses kompilasi dan menjalankan program, README.md sebagai dokumentasi umum proyek, serta lexer_app.exe sebagai hasil build program. Folder src berisi seluruh source code utama, yaitu main.cpp sebagai pengatur alur eksekusi program, lexer.hpp sebagai deklarasi struktur data dan antarmuka lexer, serta lexer.cpp sebagai implementasi proses lexical analysis berbasis DFA. Folder test digunakan untuk kebutuhan pengujian, yang dipisahkan menjadi test/input untuk menyimpan file masukan seperti sample.txt dan variasi kasus uji lain, serta test/output untuk menyimpan hasil tokenisasi dari setiap input. Sementara itu, folder doc berisi laporan tiap milestone

III.4 Implementasi DFA

III.4.1 State Definition

Nama State	Tipe State	Karakter Masukan	Deskripsi / Output Token
Start	Start	-	State awal program sebelum membaca karakter.
q_identifier	Non-Final	Huruf / Angka	Membaca kombinasi huruf dan angka secara berulang. Menghasilkan token ident.
q_p	Non-Final	p	Membaca huruf 'p' awalan keyword program / procedure.
q_pr	Non-Final	r	Membaca huruf 'r' setelah 'p'.
q_pro	Non-Final	o	Membaca huruf 'o' setelah 'pr'. Titik percabangan.
q_prog	Non-Final	g	Membaca huruf 'g' dari percabangan pro.
q_progr	Non-Final	r	Membaca huruf 'r' setelah 'prog'.
q_progra	Non-Final	a	Membaca huruf 'a' setelah 'progr'.
q_programsy	Final	m	Membaca huruf 'm'. Menghasilkan token programsy.
q_proc	Non-Final	c	Membaca huruf 'c' dari percabangan pro.
q_proce	Non-Final	e	Membaca huruf 'e' setelah 'proc'.
q_proced	Non-Final	d	Membaca huruf 'd' setelah 'proce'.
q_procedu	Non-Final	u	Membaca huruf 'u' setelah 'proced'.
q_procedur	Non-Final	r	Membaca huruf 'r' setelah 'procedu'.
proceduresy	Final	e	Membaca huruf 'e'. Menghasilkan token proceduresy.
q_c	Non-Final	c	Membaca huruf 'c' awalan keyword const / case.

q_co	Non-Final	o	Membaca huruf 'o' dari percabangan c.
q_con	Non-Final	n	Membaca huruf 'n' setelah 'co'.
q_cons	Non-Final	s	Membaca huruf 's' setelah 'con'.
q_constsy	Final	t	Membaca huruf 't'. Menghasilkan token constsy.
q_ca	Non-Final	a	Membaca huruf 'a' dari percabangan c.
q_cas	Non-Final	s	Membaca huruf 's' setelah 'ca'.
casetsy	Final	e	Membaca huruf 'e'. Menghasilkan token casesy.
q_t	Non-Final	t	Membaca huruf 't' awalan keyword type / then / to.
q_ty	Non-Final	y	Membaca huruf 'y' dari percabangan t.
q_typ	Non-Final	p	Membaca huruf 'p' setelah 'ty'.
q_typesy	Final	e	Membaca huruf 'e'. Menghasilkan token typesy.
q_th	Non-Final	h	Membaca huruf 'h' dari percabangan t.
q_the	Non-Final	e	Membaca huruf 'e' setelah 'th'.
q_thentsy	Final	n	Membaca huruf 'n'. Menghasilkan token thensy.
q_tosy	Final	o	Membaca huruf 'o' dari percabangan t. Menghasilkan token tosy.
q_v	Non-Final	v	Membaca huruf 'v' awalan keyword var.
q_va	Non-Final	a	Membaca huruf 'a' setelah 'v'.
q_varsy	Final	r	Membaca huruf 'r'. Menghasilkan token varsy.
q_f	Non-Final	f	Membaca huruf 'f' awalan keyword function / for.
q_fu	Non-Final	u	Membaca huruf 'u' dari percabangan f.
q_fun	Non-Final	n	Membaca huruf 'n' setelah 'fu'.
q_func	Non-Final	c	Membaca huruf 'c' setelah 'fun'.
q_funct	Non-Final	t	Membaca huruf 't' setelah 'func'.
q_functi	Non-Final	i	Membaca huruf 'i' setelah 'funct'.
q_functio	Non-Final	o	Membaca huruf 'o' setelah 'functi'.
q_functionsy	Final	n	Membaca huruf 'n'. Menghasilkan token functionsy.
q_fo	Non-Final	o	Membaca huruf 'o' dari percabangan f.
q_forsy	Final	r	Membaca huruf 'r'. Menghasilkan token forsy.
q_r	Non-Final	r	Membaca huruf 'r' awalan keyword record / repeat.
q_re	Non-Final	e	Membaca huruf 'e' setelah 'r'. Titik percabangan.

q_rec	Non-Final	c	Membaca huruf 'c' dari percabangan re.
q_reco	Non-Final	o	Membaca huruf 'o' setelah 'rec'.
q_recor	Non-Final	r	Membaca huruf 'r' setelah 'reco'.
recordsy	Final	d	Membaca huruf 'd'. Menghasilkan token recordsy.
q_rep	Non-Final	p	Membaca huruf 'p' dari percabangan re.
q_repe	Non-Final	e	Membaca huruf 'e' setelah 'rep'.
q_repea	Non-Final	a	Membaca huruf 'a' setelah 'repe'.
repeatsy	Final	t	Membaca huruf 't'. Menghasilkan token repeatsy.
q_n	Non-Final	n	Membaca huruf 'n' awalan keyword not.
q_no	Non-Final	o	Membaca huruf 'o' setelah 'n'.
q_notsy	Final	t	Membaca huruf 't'. Menghasilkan token notsy.
q_d	Non-Final	d	Membaca huruf 'd' awalan keyword div / do / downto.
q_di	Non-Final	i	Membaca huruf 'i' dari percabangan d.
q_idiv	Final	v	Membaca huruf 'v'. Menghasilkan token idiv.
q_do	Non-Final	o	Membaca huruf 'o' dari percabangan d. (Bisa menjadi token dosy).
q_dow	Non-Final	w	Membaca huruf 'w' setelah 'do'.
q_down	Non-Final	n	Membaca huruf 'n' setelah 'dow'.
q_downt	Non-Final	t	Membaca huruf 't' setelah 'down'.
downtosy	Final	o	Membaca huruf 'o'. Menghasilkan token downtosy.
q_o	Non-Final	o	Membaca huruf 'o' awalan keyword or / of.
q_orsy	Final	r	Membaca huruf 'r' dari percabangan o. Menghasilkan token orsy.
q_ofsy	Final	f	Membaca huruf 'f' dari percabangan o. Menghasilkan token ofsy.
q_m	Non-Final	m	Membaca huruf 'm' awalan keyword mod.
q_mo	Non-Final	o	Membaca huruf 'o' setelah 'm'.
q_imod	Final	d	Membaca huruf 'd'. Menghasilkan token imod.
q_a	Non-Final	a	Membaca huruf 'a' awalan keyword and / array.
q_an	Non-Final	n	Membaca huruf 'n' dari percabangan a.
q_andsy	Final	d	Membaca huruf 'd'. Menghasilkan token andsy.
q_ar	Non-Final	r	Membaca huruf 'r' dari percabangan a.
q_arr	Non-Final	r	Membaca huruf 'r' setelah 'ar'.

q_arra	Non-Final	a	Membaca huruf 'a' setelah 'arr'.
q_arraysy	Final	y	Membaca huruf 'y'. Menghasilkan token arraysy.
q_b	Non-Final	b	Membaca huruf 'b' awalan keyword begin.
q_be	Non-Final	e	Membaca huruf 'e' setelah 'b'.
q_beg	Non-Final	g	Membaca huruf 'g' setelah 'be'.
q_begi	Non-Final	i	Membaca huruf 'i' setelah 'beg'.
q_beginsy	Final	n	Membaca huruf 'n'. Menghasilkan token beginsy.
q_w	Non-Final	w	Membaca huruf 'w' awalan keyword while.
q_wh	Non-Final	h	Membaca huruf 'h' setelah 'w'.
q_whi	Non-Final	i	Membaca huruf 'i' setelah 'wh'.
q_whil	Non-Final	l	Membaca huruf 'l' setelah 'whi'.
q_whilesy	Final	e	Membaca huruf 'e'. Menghasilkan token whilesy.
q_i	Non-Final	i	Membaca huruf 'i' awalan keyword if.
q_ifsy	Final	f	Membaca huruf 'f'. Menghasilkan token ifsy.
q_u	Non-Final	u	Membaca huruf 'u' awalan keyword until.
q_un	Non-Final	n	Membaca huruf 'n' setelah 'u'.
q_unt	Non-Final	t	Membaca huruf 't' setelah 'un'.
q_unti	Non-Final	i	Membaca huruf 'i' setelah 'unt'.
q_untilsy	Final	l	Membaca huruf 'l'. Menghasilkan token untilsy.
q_e	Non-Final	e	Membaca huruf 'e' awalan keyword else / end.
q_el	Non-Final	l	Membaca huruf 'l' dari percabangan e.
q_els	Non-Final	s	Membaca huruf 's' setelah 'el'.
q_elsesy	Final	e	Membaca huruf 'e'. Menghasilkan token elsesy.
q_en	Non-Final	n	Membaca huruf 'n' dari percabangan e.
q_endsy	Final	d	Membaca huruf 'd'. Menghasilkan token endsy.
q_intcon	Final	Angka	Membaca digit angka, looping jika angka diteruskan. Menghasilkan token intcon.
q_(angka).	Non-Final	.	Membaca tanda titik '.' setelah angka untuk indikasi desimal.
q_realcon	Final	Angka	Membaca digit angka setelah titik, looping jika angka diteruskan. Menghasilkan token realcon.
q_'	Non-Final	‘	Membaca tanda petik tunggal pembuka teks.

q_'(huruf)	Non-Final	Any-char	Membaca sembarang karakter pertama di dalam tanda petik.
q_char	Final	‘	Membaca petik tunggal penutup tepat setelah 1 karakter. Menghasilkan token charcon.
q_'(string)	Non-Final	Huruf	Membaca karakter tambahan (looping) setelah karakter pertama.
q_string	Final	‘	Membaca petik tunggal penutup setelah string panjang. Menghasilkan token string.
q_{	Non-Final	{	Membaca awal penanda komentar (kurung kurawal buka).
q_(*	Non-Final	* atau Any dari q_(*(comment(* selain * dan)	Membaca bintang setelah membaca kurung buka (*left parenthesis*).
q_(*(comment)*	Non-Final	*	Membaca bintang setelah membaca isi komentar.
q_comment	Final	} atau)	Membaca penutup komentar. Menghasilkan token comment (diabaikan program).
q_plus	Final	+	Membaca operator tambah. Menghasilkan token plus.
q_minus	Final	-	Membaca operator kurang. Menghasilkan token minus.
q_times	Final	*	Membaca operator kali. Menghasilkan token times.
q_rdiv	Final	/	Membaca operator bagi riil. Menghasilkan token rdiv.
q_one_equal	Final	=	Membaca tanda sama dengan tunggal. Menghasilkan token eql.
q_equal	Final	=	Membaca tanda sama dengan kedua (==). Menghasilkan token eql.
q_lss	Final	<	Membaca tanda kurang dari. Menghasilkan token lss.
q_leq	Final	=	Membaca sama dengan setelah kurang dari (<=). Menghasilkan token leq.
q_neq	Final	>	Membaca lebih dari setelah kurang dari (<>). Menghasilkan token neq.
q_gtr	Final	>	Membaca tanda lebih dari. Menghasilkan token gtr.
q_geq	Final	=	Membaca sama dengan setelah lebih dari (>=). Menghasilkan token geq.
q_lparent	Final	(	Membaca kurung buka biasa. Menghasilkan token lparent.
q_rparent	Final	)	Membaca kurung tutup biasa. Menghasilkan token rparent.

q_lbrack	Final	[	Membaca kurung siku buka. Menghasilkan token lbrack.
q_rbrack	Final	]	Membaca kurung siku tutup. Menghasilkan token rbrack.
q_colon	Final	:	Membaca tanda titik dua. Menghasilkan token colon.
becomes	Final	=	Membaca sama dengan setelah titik dua (:=). Menghasilkan token becomes.
q_comma	Final	,	Membaca tanda koma. Menghasilkan token comma.
q_semicolon	Final	;	Membaca titik koma. Menghasilkan token semicolon.
q_period	Final	.	Membaca tanda titik. Menghasilkan token period.
q_reset	Non-Final	newline	Ketika satu token telah dihasilkan, akan transisi ke reset sebagai perantara ke start state.

III.4.2 Implementasi diagram DFA

1. Gambaran Umum DFA

   DFA memiliki satu state awal (Start) yang menjadi titik masuk setiap kali lexer siap membaca token baru. Dari state awal, karakter pertama yang dibaca menentukan "jalur" mana yang akan diambil. Setiap jalur berakhir di state final yang merepresentasikan satu token.

   Ketika telah mencapai state final dan menghasilkan token, DFA kembali ke state awal melalui state q_reset saat newline untuk membaca token berikutnya, sehingga lexer bisa memproses seluruh source code secara berulang.

2. Kelompok Transisi

   a. Identifier dan Keyword

      Pada Start, jika karakter pertama adalah huruf, DFA masuk ke state khusus per huruf awal (misal huruf p → q_p, huruf b → q_b, dst.). Dari setiap state huruf awal itu, DFA mencoba mencocokkan karakter berikutnya satu per satu mengikuti ejaan keyword seperti pada keyword program:

      Start, p → q_p, r → q_pr, o → q_pro, g → q_prog, r → q_progr, a → q_progra, m → q_programsy (FINAL)

Jika di tengah jalan ada karakter yang tidak sesuai ejaan keyword (misal setelah pro bukan g atau c), DFA bercabang ke q_identifier state membaca terus kombinasi huruf dan angka hingga habis, lalu menghasilkan token ident. Percabangan terjadi pada state intermediate, misalnya q_pro membaca karakter berikutnya, jika g lanjut ke jalur program, jika c lanjut ke jalur procedure.

State q_identifier juga berfungsi sebagai fallback universal: kapanpun DFA sedang di jalur keyword manapun lalu menerima huruf/angka yang tidak sesuai, DFA loncat ke q_identifier dan terus membaca hingga bukan huruf/angka, menghasilkan ident.

b. Konstanta Angka

Pada Start, karakter digit memicu masuk ke q_intcon yang looping selama karakter berikutnya masih digit. Jika setelah digit ditemukan titik yang diikuti digit lagi, DFA berpindah ke q_(angka). dan ke q_realcon (looping digit lagi) dan menghasilkan token realcon.

Jika titik tidak diikuti digit (atau tidak ada titik sama sekali), q_intcon adalah state final dan menghasilkan token intcon. DFA selalu lookahead satu karakter setelah titik untuk memastikan itu bukan operator period yang kebetulan mengikuti angka.

c. Karakter dan String

Tanda petik tunggal (‘) membuka dua kemungkinan yang baru ditentukan setelah membaca isi di dalamnya. Setelah petik pembuka, DFA membaca karakter pertama ke q_'(huruf). Jika langsung bertemu petik penutup maka menuju q_char dan menjadi token charcon. Jika ada karakter tambahan, DFA akan masuk ke q_'(string) yang looping hingga menemukan petik penutup masuk ke q_string dan jadi token string.

d. Komentar

Terdapat dua variasi sintaks komentar yang ditangani, komentar kurung kurawal dan komentar kurung-bintang. State q_comment adalah final, tetapi hasilnya diabaikan (tidak masuk output token), DFA langsung kembali ke Start untuk token berikutnya. Untuk komentar (* *), perlu state

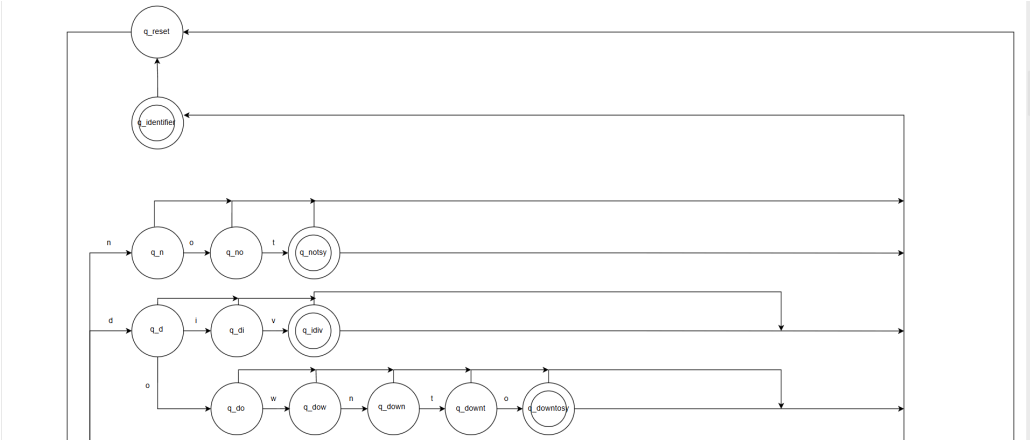
perantara q_(*comment)* khusus untuk mendeteksi pola *) penutup, karena * saja belum tentu penutup.

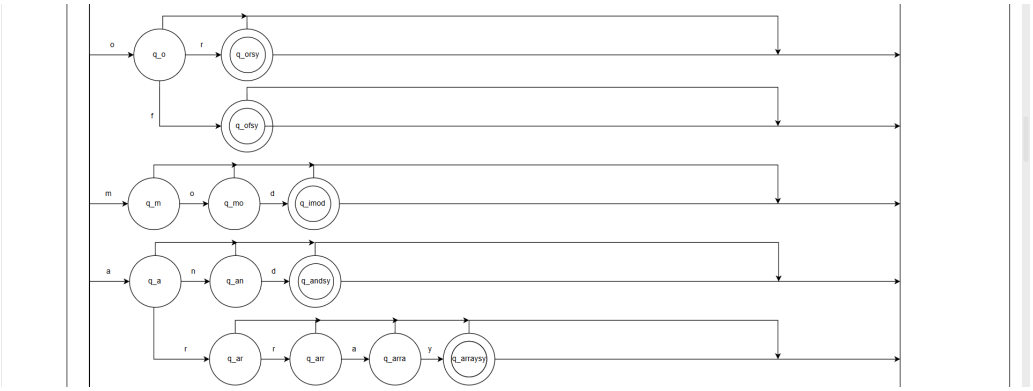
e. Operasi Satu Karakter

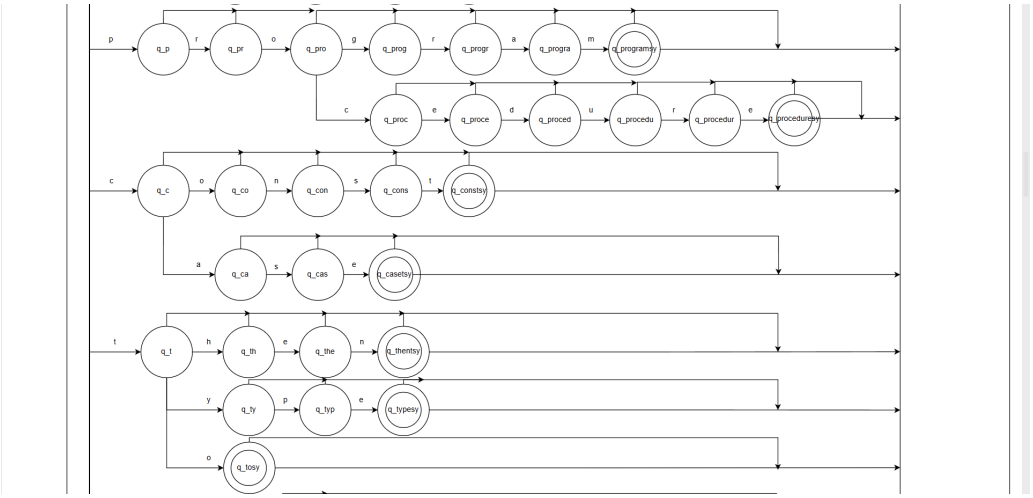
Transisi paling sederhana, langsung dari Start ke state final dalam satu langkah, seperti q_plus, q_minus, q_times, dll.

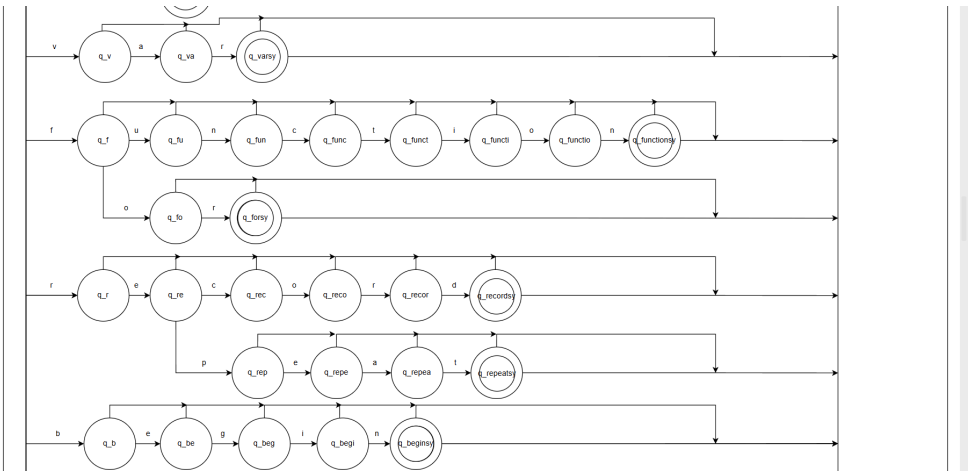
f. Operator Multi-Karakter

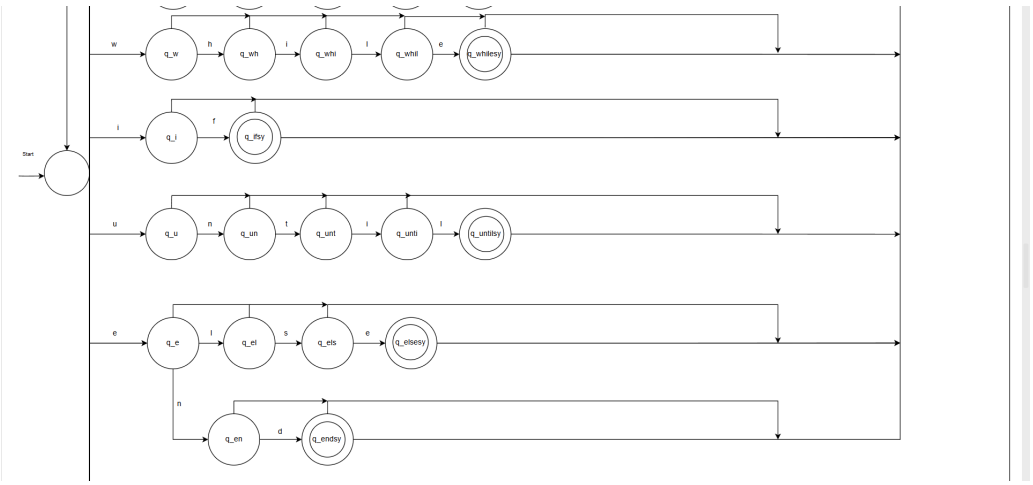
Beberapa operator memerlukan membaca karakter berikutnya untuk menentukan token yang tepat, yaitu operator < (lss), <= (leq), <> (neq), > (gtr), >= (geq), == (eql), : (colon), := (becomes), kurung buka.

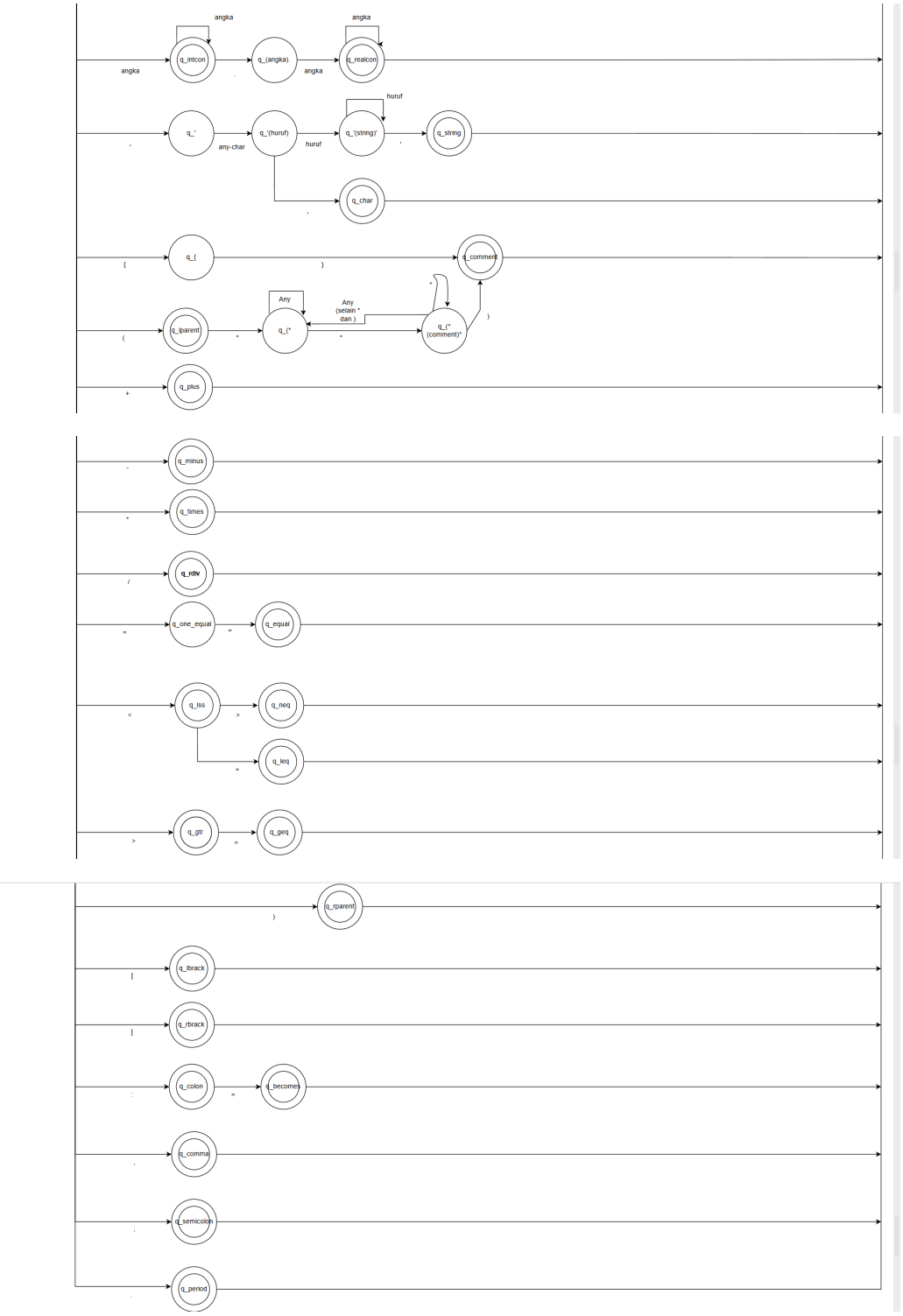





p
q_p
r
q_pr
o
q_pro
g
q_prog
r
q_progr
a
q_progra
m
q_programsy
c
q_proc
e
q_proce
d
q_proced
u
q_procedu
r
q_procedur
e
q_procedursy
c
q_c
o
q_co
n
q_con
s
q_cons
t
q_constsy
a
q_ca
s
q_cas
e
q_casesy
t
q_t
h
q_th
e
q_the
n
q_thentsy
y
q_ty
p
q_typ
e
q_typesy
o
q_tosy


v
q_v
a
q_va
r
q_varsy
f
q_f
u
q_fu
n
q_fun
c
q_func
t
q_funct
i
q_functi
o
q_functio
n
q_functionsy
o
q_fo
r
q_forsy
r
q_r
e
q_re
c
q_rec
o
q_reco
r
q_recor
d
q_recordsy
p
q_rep
e
q_repe
a
q_repea
t
q_repeatsy
b
q_b
e
q_be
g
q_beg
i
q_begi
n
q_beginsy


w
q_w
h
q_wh
i
q_whi
l
q_whil
e
q_whilesy
i
q_i
f
q_ifsy
Start
u
q_u
n
q_un
t
q_unt
i
q_unti
l
q_untilsy
e
q_e
l
q_el
s
q_els
e
q_elsesy
n
q_en
d
q_endsy



Gambar Diagram DFA

BAB IV

PENGUJIAN

IV.1 Uji 1

File Input Uji 1:

```
program Hello;
var
    a, b: integer;
begin
    a := 5;
    b := a + 10;
    writeln('Result = ', b);
end.
```

File Output Uji 1:

```
programsy
ident (Hello)
semicolon
varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon
beginsy
```

```
ident (a)
becomes
intcon (5)
semicolon
ident (b)
becomes
ident (a)
plus
intcon (10)
semicolon
ident (writeln)
lparent
string ('Result = ')
comma
ident (b)
rparent
semicolon
endsy
Period
```

Analisis Hasil : Program berhasil mengidentifikasi seluruh token yang ada di file input mulai dari identifier, string, intcon, semicolon, colon ,comma, dan lain lain

IV.2 Uji 2

File Input Uji 2:

```
program Rapat;
var
```

```
    a, b: integer;
begin
    if (a <= 10) and (b <> 5) then
    a := b + 20;
end.
```

File Output Uji 2:

```
programsy
ident (Rapat)
semicolon
varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon
beginsy
ifsy
lparent
ident (a)
leq
intcon (10)
rparent
andsy
lparent
```

```
ident (b)
neq
intcon (5)
rparent
thensy
ident (a)
becomes
ident (b)
plus
intcon (20)
semicolon
endsy
period
```

Analisis Hasil : Program berhasil identifikasi seluruh token termasuk operator operator logika seperti kurang dari sama dengan (<=), tidak sama dengan (<>), dan operator operator lain

IV.3 Uji 3

File Input Uji 3:

```
program StringTest;
const
    huruf := 'a';
var
    arrayzz;
    pesan: string;
```

```
begin
    pesan := 'begin := 5; {bukan komentar}';
    huruf := '';
    {kalo ini baru komentar}
    (*ini juga*)
end.
```

File Output Uji 3:

```
programsy
ident (Rapat)
semicolon
varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon
beginsy
ifsy
lparent
ident (a)
leq
intcon (10)
rparent
andsy
```

```
lparent
ident (b)
neq
intcon (5)
rparent
thensy
ident (a)
becomes
ident (b)
plus
intcon (20)
semicolon
endsy
period
```

Analisis Hasil : Program bisa membaca token token seperti charcon dan string yang diapit oleh dua petik (‘’), program ini juga membaca token token seperti komentar yang diapit oleh kurung biasa dengan bintang (**) maupun kurung kurawal.

IV.4 Uji 4

File Input Uji 4:

```
program Angka;


var
    x, y: real;
    z: integer;
```

```
begin
    x := 3.14159;
    y := 007;
    z := 5;
end.
```

File Output Uji 4:

```
programsy
ident (Angka)
semicolon
varsy
ident (x)
comma
ident (y)
colon
ident (real)
semicolon
ident (z)
colon
ident (integer)
semicolon
beginsy
ident (x)
becomes
realcon (3.14159)
semicolon
```

```
ident (y)
becomes
intcon (007)
semicolon
ident (z)
becomes
intcon (5)
semicolon
endsy
period
```

Analisis Hasil : Program berhasil membaca dan memproses file input dengan baik. Program berhasil mengidentifikasi tipe data seperti integer dan real. Program juga berhasil mengidentifikasi identifier. Selain itu, program juga berhasil mengidentifikasi angka-angka dan menampilkannya sebagai intcon untuk integer, dan realcon untuk bilangan real pada output.

IV.5 Uji 5

File Input Uji 3:

```
program nanggung;


begin
    writeln('hello
end.
```

File Output Uji 3:

```
programsy
ident (nanggung)
semicolon
```

```
beginsy
ident (writeln)
lparent
unknown (string tidak ditutup sebelum akhir file)
```

Analisis Hasil :Program berhasil mengidentifikasi dan handle sebuah unknown token yaitu string yang tidak tertutup oleh 2 kurung dan menampilkannya di output file, selain unknown token, token yang lain berhasil diidentifikasi oleh program

IV.6 Uji 6

File Input Uji 6:

```
program aneh;

var
    a, b: integer;

begin
    a @ b;
    writeln('Aku cinta asisten ''hehe''');
End.
```

File Output Uji 6:

```
programsy
ident (aneh)
semicolon
varsy
ident (a)
```

```
comma
ident (b)
colon
ident (integer)
semicolon
beginsy
ident (a)
unknown (@)
ident (b)
semicolon
ident (writeln)
lparent
string ('Aku cinta asisten 'hehe'')
rparent
semicolon
endsy
period
```

Analisis Hasil : Program berhasil memproses file input dengan baik. Ketika terdapat token yang tidak dikenali, dalam input ini adalah karakter '@', program berhasil meng-*handle*-nya dengan baik. Program akan menampilkan keterangan `unknown (@)` di output dan meneruskan memproses file. Selain itu, program juga berhasil meng-*handle* input string yang memiliki *escape character* di dalamnya. Di sini, *escape character* pada string adalah input tanda petik satu (') sebanyak dua kali. Ketika terdapat dua tanda petik satu di dalam string, nantinya akan ditampilkan sebagai satu tanda petik di output. Contoh: `string ('Aku cinta asisten 'hehe'')` untuk input: `'Aku cinta asisten ''hehe'''`

IV.7 Uji 7

File Input Uji 7(File Kosong):

File Output Uji 7(Output Kosong):

Analisis Hasil : Program berhasil meng-*handle* file kosong dengan baik. Ketika file kosong dijadikan sebagai file input, nantinya akan muncul file output yang juga merupakan file kosong.

IV.8 Uji 8

File Input Uji 8:

```
program negative;


var
    a, b: integer;


begin
    a := -12;
    b = 2;
    writeln('a adalah bilangan negatif');
    writeln('b bukan bilangan negatif');
end.
```

File Output Uji 8:

```
programsy
ident (negative)
semicolon
```

```
varsy
ident (a)
comma
ident (b)
colon
ident (integer)
semicolon
beginsy
ident (a)
becomes
intcon (-12)
semicolon
ident (b)
unknown (=)
intcon (2)
semicolon
ident (writeln)
lparent
string ('a adalah bilangan negatif')
rparent
semicolon
ident (writeln)
lparent
string ('b bukan bilangan negatif')
rparent
semicolon
```

```
endsy
period
```

Analisis Hasil : Program berhasil membaca token intcon yang positif (intcon 2) maupun negatif (intcon -12) pada file input, dan juga dapat menghandle unknown token seperti satu sama dengan (=) tanpa menghentikan pembacaan file melainkan melanjutkan pembacaan file sampai akhir.

BAB V

KESIMPULAN DAN SARAN

V.1 Kesimpulan

Dari Milestone 1 Tugas Besar Teori Bahasa Formal dan Automata kali ini, kami dapat mengambil beberapa kesimpulan, kami dapat mengimplementasi algoritma lexical analyzer pada program ini membuat mengidentifikasi token token yang sesuai dengan spesifikasi, pada Milestone 1 ini juga kami telah implementasi diagram Deterministic Finite Automata (DFA) yang menggambarkan proses transisi dari start state ke final state pada pembacaan token. Semoga dari Milestone 1 ini kami mendapat pelajaran berharga dan bisa membuat compiler yang menyaingi c++.

V.2 Saran

Pelaksanaan Milestone 1 Tugas Besar Teori Bahasa Formal dan Automata di Semester II (Genap) Tahun 2025/2026 merupakan pengalaman yang sangat berharga bagi kami. Dari pengalaman ini, kami ingin berbagi beberapa saran kepada pembaca yang mungkin akan menghadapi tugas serupa di masa depan:

Yusuf : Semoga milestone selanjutnya QNA nya ga sebanyak milestone 1

Farrel : QnA banyak kali bang T-T, jadi gk tenang.

Nathanael : Spesifikasi dalam bentuk komik kak biar asik.

Azam : Aku cinta asisten, makasih asisten.

BAB VI

LAMPIRAN

VI.1 Lampiran Link Repository Github.

Link : https://github.com/ucupsss/SMG-Tubes-IF2224-2026

VI.2 Lampiran Link Workspace Diagram Transisi DFA

Link:

- draw.io: https://drive.google.com/file/d/1lG9_BzxuQpJIjzbcltqv6kWRYYivbf8z/view?usp=sharing
- HTML (buka pada browser untuk diagram yang lebih jelas: https://drive.google.com/file/d/1mT7fSK8AOM5rlW5PkUEm_ShUvP4vhaOt/view?usp=sharing

VI.3 Pembagian

Nama	Tugas
Yusuf Faishal Listyardi	Membuat DFA, Membuat code program, Membuat Laporan
Farrell Limjaya	Membuat DFA, Membuat code program, Membuat Laporan
Nathanael Gunawan	Membuat DFA, Membuat code program, Membuat Laporan
A. Fawwaz Azam Wicaksono	Membuat DFA, Membuat code program, Membuat Laporan